A Finite-State Proof of the Well-Definedness of a Perturbed Hofstadter Sequence

Finite constraint reduction and complete critical-core verification

Marco Mantovanelli*

Abstract

We prove that the perturbed Hofstadter-type sequence defined by

$$Q(1) = 1, \quad Q(2) = 1, \quad Q(n) = Q(n - Q(n-1)) + Q(n - Q(n-2)) + (-1)^n$$

is well-defined for all $n \geq 1$. That is, all recursive arguments remain positive and the recursion is globally well-defined.

The proof proceeds by reducing the infinite recursion to a finite combinatorial constraint system. We introduce a symbolic encoding of local configurations and enumerate a finite set of admissible contexts together with a compatibility relation capturing all valid local transitions. Global realizations of the recursion correspond to solutions of this finite system.

A key structural ingredient is a decomposition of all valid assignments into two global modes, which allows the reduction of potential obstructions to a finite critical core. A complete finite verification then excludes all obstructions and establishes global well-definedness.

More broadly, the argument shows that certain meta-Fibonacci recursions, despite their non-local definition, admit a finite-state description whose consistency can be decided by exhaustive combinatorial analysis.

Keywords: meta-Fibonacci sequence, Hofstadter sequence, perturbed recursion, well-definedness, finite-state method, constraint satisfaction, combinatorial dynamics, recursive sequences

MSC 2020: 11B37, 05C20, 68R10, 05A15

1 Introduction

Meta-Fibonacci recursions form a class of sequences in which the arguments of the recursion depend on previously computed values. Among the most prominent examples is Hofstadter's Q-sequence, introduced in [11], defined by

$$Q(n) = Q(n - Q(n-1)) + Q(n - Q(n-2)).$$

Despite its simple definition, the sequence exhibits highly nontrivial behavior, and even basic structural properties remain difficult to analyze. In particular, the question whether the sequence is

*Independent Researcher. marco@mantovanelli.de

well-defined for all n has remained notoriously difficult; see, for example, [15, 16, 3, 1, 17] for related discussions and variants.

Recent work has further highlighted the diversity of behaviors that can arise in Hofstadter-type recursions. In particular, Fox [8, 7, 9] demonstrates that such recursions can exhibit a wide range of behaviors depending on the choice of initial conditions, including structured, periodic, and chaotic regimes.

At the same time, it is known that, in general, questions about nested recurrence relations can be undecidable [6], which underscores the inherent difficulty of establishing global properties such as well-definedness.

In this paper, we consider the perturbed Hofstadter-type recursion

$$Q(1) = 1, \quad Q(2) = 1, \quad Q(n) = Q(n - Q(n-1)) + Q(n - Q(n-2)) + (-1)^n.$$

The additional alternating term introduces a controlled perturbation of the classical recursion while preserving its essential nonlocal character.

This perturbed model was introduced and studied in [13], where strong empirical and structural evidence for regularity and self-similarity was obtained. In particular, the numerical and symbolic analysis in [13] suggests that the sequence remains globally well-behaved despite the presence of alternating forcing.

A fundamental question for such recursions is *well-definedness*: whether the recursion produces a value for every $n \geq 1$, equivalently, whether all recursive arguments remain positive throughout the evolution of the sequence. The main result of the present paper is a complete resolution of this question for the perturbed sequence.

Theorem 1. *The perturbed Hofstadter-type sequence*

$$Q(n) = Q(n - Q(n-1)) + Q(n - Q(n-2)) + (-1)^n$$

with initial values $Q(1) = Q(2) = 1$ *is well-defined for all* $n \geq 1$.

1.1 Method of proof

The proof proceeds by replacing the infinite recursive structure with a finite combinatorial model. In Section 2 we explain the reduction of the recursion to a finite symbolic system. Section 3 introduces the local contexts that encode the relevant neighborhood data, and Section 4 constructs the compatibility graph governing local propagation. In Section 5 we formulate the notion of global assignments and recast the problem as a finite constraint satisfaction problem.

The key structural ingredients appear in the next three sections. Section 6 establishes rigidity at debt-2 contexts. Section 7 proves the two-mode decomposition of valid assignments, and Section 8 introduces the associated support sets, which describe all locally admissible values in each mode.

The obstruction analysis occupies the final part of the paper. In Section 9 we reduce the obstruction problem to a finite critical core. Section 10 analyzes the induced structure on this core, and Section 11 carries out the finite verification showing that every subset of the critical core admits a valid assignment. Section 12 then combines all previous reductions to complete the proof of the main theorem.

The appendices contain the machine-checkable certificate for the critical-core verification, the full automaton data, and the compressed transition relation used in the computation. A complete computational implementation of the finite verification is provided in a publicly available repository [14].

1.2 Position within the literature

The present argument combines ideas from the study of meta-Fibonacci recursions with methods that are more commonly associated with finite symbolic dynamics and finite constraint systems. In this sense, the proof shows that a nontrivial self-referential recursion can nevertheless admit a finite-state description at the level of local behavior; compare [12, 10] for general background on symbolic systems and graph-homomorphism viewpoints. Recent work on Hofstadter-type and related meta-Fibonacci structures also points toward a broad variety of emergent combinatorial behaviors; see, for example, [2, 18, 5].

From a broader perspective, the result supports the idea that certain recursively defined sequences, although globally difficult to analyze directly, may be controlled through a finite local model together with an explicit obstruction analysis.

Scope and limitations. The focus of the present work is exclusively on the question of well-definedness, namely, the existence of a globally consistent realization of the recursion. While the finite-state reduction developed here provides a complete solution to this problem, it is not intended to capture the full combinatorial complexity of the sequence. In particular, we do not address the precise asymptotic rate of convergence of $Q(n)/n$ to $1/2$, nor do we claim that the finite symbolic model fully reflects the deeper combinatorial structures that may govern this convergence. It is conceivable that finer analyses involve richer objects (such as Catalan-type structures or lattice-path enumerations), which lie beyond the scope of the present approach.

2 Reduction to a Finite Symbolic System

The central difficulty in analyzing the recursion

$$Q(n) = Q(n - Q(n-1)) + Q(n - Q(n-2)) + (-1)^n$$

lies in its nonlocal nature: the arguments of the recursion depend on previously computed values and may range over a large portion of the past. As a result, direct inductive arguments are difficult to control.

The key idea of this paper is that, despite this nonlocal dependence, the recursion exhibits a strong form of *local regularity*. More precisely, while the numerical values of $Q(n)$ are unbounded, the *relative structure* of the recursion in a bounded neighborhood of n can be encoded using finitely many symbolic configurations.

2.1 From recursion to symbolic data

Instead of working directly with the values of $Q(n)$, we introduce a finite set of symbolic states

$$S = \{S_0, S_1, \ldots, S_7\},$$

which encode the local behavior of the recursion. Each symbolic state represents an equivalence class of local configurations that behave identically with respect to the recursive rule. We write elements of $S \times \{0, 2\}$ in the shorthand form $S_i[d]$ instead of (S_i, d).

In addition, we introduce a *debt parameter*

$$d \in \{0, 2\},$$

which accounts for the bounded discrepancy introduced by the alternating perturbation term $(-1)^n$. This parameter ensures closure of the symbolic system under propagation.

2.2 Local configurations and contexts

To capture the local structure of the recursion, we consider finite windows around a position n and encode them by *contexts*. Each context represents a class of local configurations that are indistinguishable with respect to future propagation.

The set of all admissible contexts will be denoted by Λ_{adm}. It is finite and can be computed explicitly.

2.3 Compatibility constraints

The recursion induces local compatibility conditions between neighboring contexts. These constraints define a directed relation

$$\Psi \subseteq \Lambda_{\mathrm{adm}} \times \Lambda_{\mathrm{adm}},$$

which specifies which contexts may follow each other in a valid configuration. We identify Ψ with the edge set of a directed graph on Λ_{adm}.

Thus, the infinite recursion is reduced to a finite combinatorial system consisting of:

- a finite set of contexts Λ_{adm},
- a finite set of symbolic states,
- a finite compatibility relation Ψ.

2.4 Global consistency problem

A realization of the recursion corresponds to a global assignment of symbolic states to contexts that respects all compatibility constraints.

The problem of well-definedness is therefore equivalent to the following finite statement:

Every locally admissible configuration extends to a globally consistent assignment.

In the subsequent sections, we make this reduction precise and analyze the resulting finite system in detail.

3 Local Contexts

We now introduce the precise formal objects that encode local configurations of the recursion.

3.1 Definition of contexts

Definition 1. *A* context *is a tuple*

$$\lambda = (p, R, i, \pi, d),$$

where:

- *p is a finite pattern describing symbolic states in a fixed local window,*

- $R \in \{R_1, R_2, R_3\}$ *specifies the recursion regime,*
- i *is a distinguished position within the pattern,*
- $\pi \in \{0, 1\}$ *is the parity of the index,*
- $d \in \{0, 2\}$ *is the debt parameter.*

We will also use an expanded notation in which the pattern p is written explicitly and the components are labeled:

$$(p, R, i, \pi, d) = (p_1, p_2, p_3, R, \mathrm{pos} = i, \mathrm{parity} = \pi, \mathrm{debt} = d).$$

Each context represents a local configuration centered at a position n, together with sufficient information to determine how the recursion propagates to neighboring positions.

3.2 Admissibility

Not every tuple (p, R, i, π, d) corresponds to a valid local configuration.

Definition 2. *A context λ is* admissible *if there exists a realization of the recursion in which the symbolic data encoded by λ is consistent with the recursive rule.*

Admissibility imposes local consistency conditions, including:

- compatibility of symbolic states within the pattern,
- consistency of recursive dependencies determined by R,
- correct propagation of parity,
- consistency of the debt parameter under recursion.

3.3 Finite enumeration

Proposition 1. *The set of admissible contexts Λ_{adm} is finite and has cardinality* 28.

We fix an enumeration

$$\Lambda_{\mathrm{adm}} = \{\lambda_0, \ldots, \lambda_{27}\},$$

and refer to contexts interchangeably by their index ("ctx i") or by λ_i.

Proof. Each component of the tuple (p, R, i, π, d) ranges over a finite set. The admissibility conditions impose only local constraints and therefore define a finite subset of the full parameter space. An explicit enumeration yields $|\Lambda_{\mathrm{adm}}| = 28$. □

3.4 Role of contexts

Contexts serve as the vertices of the compatibility graph introduced in the next section. A transition between contexts corresponds to shifting the observation window by one position in the sequence.

Thus, sequences of contexts correspond to locally consistent configurations of the recursion.

3.5 Example: A concrete context and its interpretation

We illustrate the notion of a context with an explicit example taken from the critical core.

Consider the context

$$\lambda_0 = (C, E, B, R_1, \mathrm{pos} = 1, \mathrm{parity} = 1, \mathrm{debt} = 0).$$

Interpretation. This context encodes a local configuration around some index n with the following properties:

- The pattern (C, E, B) represents a symbolic description of the values of Q in a fixed window around n.
- The rule type R_1 specifies how the recursive calls

  $$Q(n - Q(n-1)), \quad Q(n - Q(n-2))$$

  interact in this configuration.
- The position $\mathrm{pos} = 1$ indicates that the current index n corresponds to the first entry of the pattern.
- The parity $\mathrm{parity} = 1$ means that n is odd, so the perturbation contributes $(-1)^n = -1$.
- The debt parameter $d = 0$ indicates that no correction is currently carried over from previous steps.

Admissibility. The fact that $\lambda_0 \in \Lambda_{\mathrm{adm}}$ means that there exists a realization of the recursion in which a local neighborhood of some index n exhibits exactly this symbolic structure, and all recursive dependencies are satisfied.

Propagation. The compatibility graph Ψ determines which contexts may follow λ_0. In the induced structure on the critical core, we have the edge

$$\lambda_0 \to \lambda_2,$$

which means that after shifting the observation window by one position, the resulting local configuration must correspond to the context

$$\lambda_2 = (E, B, E, R_1, \mathrm{pos} = 2, \mathrm{parity} = 0, \mathrm{debt} = 0).$$

Symbolic states. The support sets at λ_0 are

$$\Sigma_A(\lambda_0) = \{S1[0]\}, \qquad \Sigma_B(\lambda_0) = \{S2[0]\}.$$

Thus, depending on the global mode, the local configuration represented by λ_0 can only realize one of two symbolic states.

Role in the global structure. This example illustrates how a single context encodes:

- local structural information about the recursion,
- constraints on neighboring configurations via Ψ,
- and mode-dependent admissible symbolic states via the support sets.

In particular, the interaction between λ_0 and its successors is one of the key mechanisms through which global constraints propagate across the system.

4 The Compatibility Graph

The finite symbolic system introduced in the previous section is governed by a local compatibility relation between contexts. This relation encodes the propagation of the recursion across neighboring positions and forms the central combinatorial object of the proof.

4.1 Definition of the compatibility relation

Let Λ_{adm} denote the set of admissible contexts. We define a directed relation

$$\Psi \subseteq \Lambda_{\mathrm{adm}} \times \Lambda_{\mathrm{adm}}$$

as follows.

Definition 3. *A pair $(\lambda, \mu) \in \Psi$ if and only if the context μ can occur immediately to the right of λ in a locally consistent realization of the recursion.*

More concretely, the relation Ψ is derived from the recursive rule

$$Q(n) = Q(n - Q(n-1)) + Q(n - Q(n-2)) + (-1)^n,$$

by enforcing that the symbolic data encoded in λ and μ are compatible with a common underlying configuration of the sequence.

This includes:

- consistency of overlapping local patterns,
- compatibility of recursive dependencies,
- agreement of parity and debt parameters under propagation.

We call μ a neighbor of λ if either $(\lambda, \mu) \in \Psi$ or $(\mu, \lambda) \in \Psi$.

4.2 Local determinism and finite propagation

A crucial feature of the system is that compatibility is *local*: the admissibility of a transition (λ, μ) depends only on the data contained in λ and μ, and not on the global configuration.

This property places the system in the general framework of *finite constraint systems* or *shifts of finite type* in symbolic dynamics (see, e.g., [12]). In this interpretation, contexts play the role of symbols, and the relation Ψ defines the allowed transitions between symbols.

Proposition 2. *The compatibility relation Ψ is finite and effectively computable. In the present case, it consists of* 34 *directed edges.*

A complete visualization of the directed compatibility graph $(\Lambda_{\mathrm{adm}}, \Psi)$ is given in Figure 1. The figure highlights the two connected components corresponding to debt 0 and debt 2, and it also makes transparent the internal regime structure coming from R_1, R_2, and R_3.

Here and below, connectedness refers to the underlying undirected graph of $(\Lambda_{\mathrm{adm}}, \Psi)$; directed propagation is considered separately along admissible directed paths.

Sketch of construction. For each pair $(\lambda, \mu) \in \Lambda_{\mathrm{adm}}^2$, one checks whether there exists a local realization of the recursion consistent with both contexts. This verification reduces to a finite set of conditions on symbolic states and parameters, and can therefore be carried out exhaustively. The resulting set of admissible pairs defines Ψ. □

4.3 Graph structure

We view $(\Lambda_{\mathrm{adm}}, \Psi)$ as a directed graph, called the *compatibility graph.* Paths in this graph correspond to sequences of contexts that can appear consecutively in a valid configuration.

Definition 4. *A path* $(\lambda_0, \lambda_1, \ldots, \lambda_k)$ *is* admissible *if*

$$(\lambda_i, \lambda_{i+1}) \in \Psi \quad \textit{for all } i.$$

Admissible paths represent locally consistent configurations of the recursion over finite intervals.

Proposition 3. *The compatibility graph decomposes into two connected components.*

Idea of proof. This follows from an explicit computation of Ψ. The decomposition reflects a global dichotomy in the behavior of the recursion, which will later be identified with the two modes of global assignments. □

4.4 Global configurations as graph colorings

A global assignment

$$f : \Lambda_{\mathrm{adm}} \to S \times \{0, 2\}$$

can be interpreted as a labeling of the vertices of the compatibility graph.

The assignment is valid if and only if it respects all edges:

$$(\lambda, \mu) \in \Psi \;\Rightarrow\; (f(\lambda), f(\mu)) \text{ is an admissible transition.}$$

Thus, valid assignments correspond to *graph homomorphisms* from the compatibility graph into a finite transition system on symbolic states.

This perspective connects the problem to classical constraint satisfaction problems (CSPs) and graph homomorphism problems (see, e.g., [10]), where one seeks assignments satisfying local compatibility constraints.

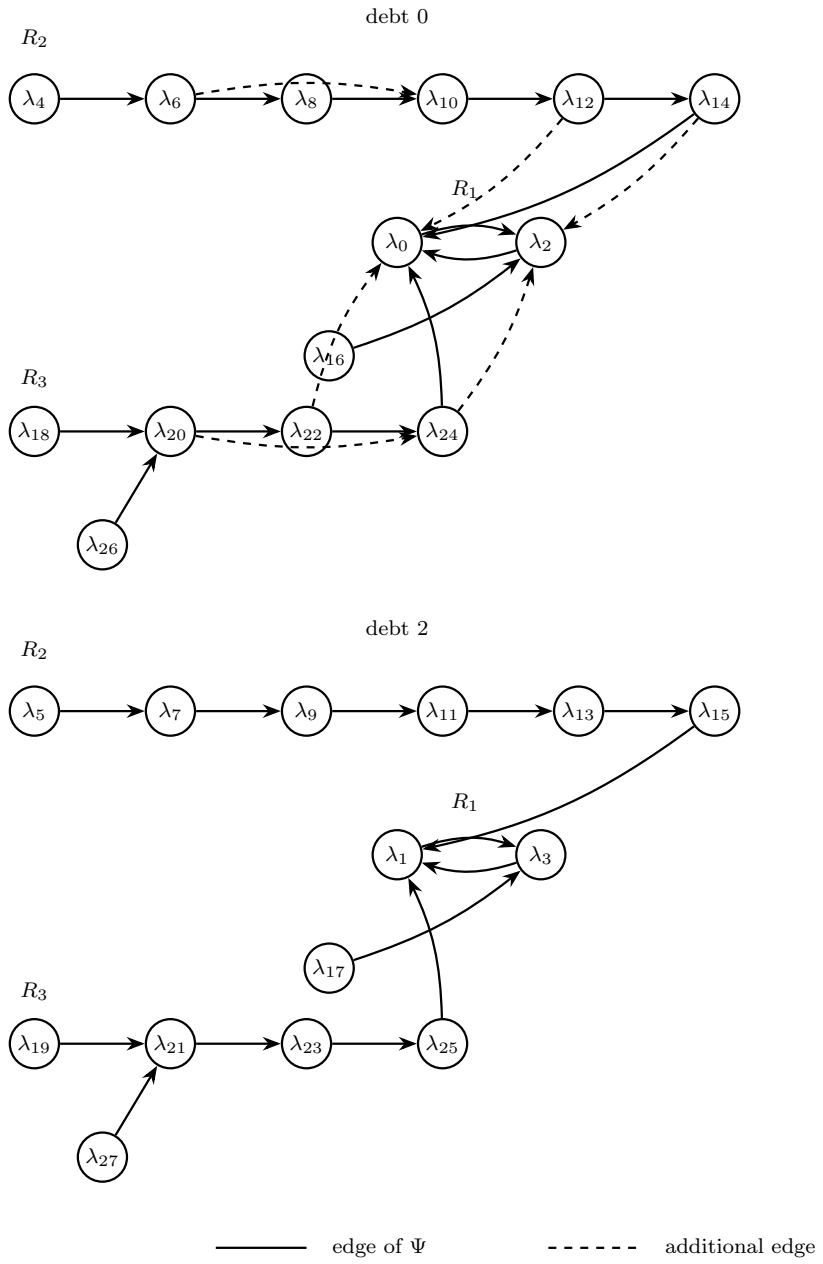

Figure 1: The compatibility graph $(\Lambda_{\text{adm}}, \Psi)$. The graph has two connected components, corresponding to debt 0 and debt 2. Solid arrows indicate the principal chain structure inside the regimes R_1, R_2, and R_3; dashed arrows indicate the additional compatibility edges from the full explicit list in Appendix B.

4.5 Relation to meta-Fibonacci dynamics

The appearance of a finite compatibility graph is a nontrivial structural feature of the recursion. While meta-Fibonacci sequences are generally considered highly irregular (see [16, 15]), the present construction shows that the perturbed sequence admits a finite-state description at the level of local behavior.

This phenomenon is closely related to the emergence of regularity patterns observed empirically in Hofstadter-type sequences and their variants [3, 19, 7].

4.6 Role in the proof

The compatibility graph provides the bridge between the infinite recursion and the finite obstruction problem.

- Local consistency is encoded by edges of Ψ.
- Global consistency corresponds to the existence of valid assignments.
- Failure of extendability corresponds to the existence of obstructions.

In the subsequent sections, we analyze the structure of this graph in detail. In particular, we show that all valid assignments fall into two global modes and that any obstruction must be supported on a small critical subset of contexts.

5 Global Assignments

Having introduced the finite set of contexts and the compatibility relation Ψ, we now formalize the notion of a global configuration of the system.

5.1 Assignments

Definition 5. *A* global assignment *is a function*

$$f : \Lambda_{\mathrm{adm}} \to S \times \{0, 2\},$$

which assigns to each admissible context a symbolic state together with a debt value.

Intuitively, an assignment specifies, for every possible local configuration, which symbolic state is realized in that configuration.

5.2 Compatibility constraints

Not every assignment corresponds to a valid configuration of the recursion. The assignments must respect the compatibility relation Ψ.

Definition 6. *A global assignment f is* valid *if for every edge $(\lambda, \mu) \in \Psi$, the pair*

$$(f(\lambda), f(\mu))$$

is an admissible transition of symbolic states.

This condition ensures that locally adjacent contexts are assigned compatible symbolic data.

5.3 Interpretation as a constraint system

The pair $(\Lambda_{\mathrm{adm}}, \Psi)$ together with the symbolic states defines a finite constraint satisfaction problem (CSP). A valid assignment is precisely a solution to this CSP.

Equivalently, one may view the problem as a graph homomorphism problem: the compatibility graph $(\Lambda_{\mathrm{adm}}, \Psi)$ must be mapped into a finite transition graph on symbolic states in a way that preserves edges (see [10]).

5.4 Relation to the recursion

The key link to the original recursion is the following principle:

> *Every realization of the recursion induces a valid assignment, and the finite symbolic system is constructed so that extendability of valid assignments is sufficient for global consistency of the recursion.*

Indeed, given a realization of the sequence $Q(n)$, each position determines a context λ together with a symbolic state. The compatibility of neighboring positions implies that the induced assignment satisfies all Ψ constraints.

Thus, the existence of suitable valid assignments provides the finite consistency criterion used in the proof of well-definedness.

Lemma 1 (Context-determined symbolic state)**.** *For every admissible context $\lambda \in \Lambda_{\mathrm{adm}}$, the symbolic state assigned by the finite-state encoding is uniquely determined by λ. Equivalently, if the same admissible context occurs at two positions in a realization of the recursion, then it induces the same element of $S \times \{0, 2\}$ at both positions.*

Proof. By construction, a context

$$\lambda = (p, R, i, \pi, d) \in \Lambda_{\mathrm{adm}}$$

contains exactly the local symbolic data used by the encoder: the local pattern p, the recursion regime R, the distinguished position i, the parity π, and the debt parameter d. The finite-state encoding was defined so that the assigned symbolic state depends only on these local parameters and on no additional global information.

Hence, whenever two occurrences of the recursion give rise to the same admissible context λ, all inputs to the encoder coincide. Therefore the output of the encoder must coincide as well. In other words, the symbolic state associated with an admissible context is a well-defined function of the context alone.

Thus the finite-state encoding determines a map

$$\mathrm{enc} : \Lambda_{\mathrm{adm}} \to S \times \{0, 2\},$$

and the symbolic state induced by any occurrence of λ is uniquely determined by λ. □

The symbolic model is constructed so that local realizability in the recursion coincides with local satisfiability in the associated finite constraint system. In particular, a locally admissible configuration is extendable in the recursion precisely when the corresponding partial assignment is extendable to a valid global assignment in the finite model. Thus, failure of recursive well-definedness gives rise to a non-extendable locally admissible configuration, i.e. to an obstruction in the finite constraint system.

Theorem 2 (Correctness of the finite-state reduction)**.** *The perturbed Hofstadter-type recursion*

$$Q(1) = 1, \qquad Q(2) = 1, \qquad Q(n) = Q(n - Q(n-1)) + Q(n - Q(n-2)) + (-1)^n$$

is well-defined for all $n \geq 1$ if and only if the associated finite constraint system has no obstruction.

Here the associated finite constraint system is the symbolic model constructed in Sections 2–4, with admissible context set Λ_{adm}, compatibility relation Ψ, and valid assignments $f : \Lambda_{\mathrm{adm}} \to S \times \{0, 2\}$.

Proof. We prove both directions.

Soundness. Assume that the recursion is well-defined for all $n \geq 1$.

Then every index n admits a genuine realization of the recursion, and hence determines a local configuration consistent with the recursive rule. By construction of the symbolic system, each such local configuration corresponds to an admissible context

$$\lambda \in \Lambda_{\mathrm{adm}},$$

together with a symbolic state in $S \times \{0, 2\}$ describing the local behavior of the recursion.

By Lemma 1, the symbolic state associated with an admissible context depends only on the context itself and not on the particular occurrence of that context in the realization.

This therefore defines a map

$$f : \Lambda_{\mathrm{adm}} \to S \times \{0, 2\},$$

obtained by assigning to each context the symbolic state realized in the sequence.

Since consecutive positions in the sequence arise from a single globally consistent realization, overlapping local windows are compatible. Therefore, for every edge

$$(\lambda, \mu) \in \Psi,$$

the pair $(f(\lambda), f(\mu))$ is an admissible symbolic transition. Hence f is a valid global assignment.

In particular, every locally admissible configuration occurring in the sequence extends to a valid global assignment.

Completeness. Assume that the associated finite constraint system has no obstruction. Equivalently, every partial assignment arising from a locally admissible configuration extends to a valid global assignment.

Suppose, for contradiction, that the recursion is not well-defined. Then there exists a smallest index $n \geq 3$ at which the recursion fails. By minimality, all values $Q(1), \ldots, Q(n-1)$ are defined.

The failure at n must therefore arise from a violation of the recursive rule, namely that at least one of the arguments

$$n - Q(n-1), \qquad n - Q(n-2)$$

is nonpositive or otherwise incompatible with the local consistency conditions imposed by the recursion.

By construction of the symbolic system, the local configuration around n determines an admissible context

$$\lambda \in \Lambda_{\mathrm{adm}},$$

and the failure of the recursion corresponds to the fact that this configuration cannot be extended to a globally consistent realization. In the symbolic model, this means precisely that the corresponding local assignment does not extend to a valid global assignment, i.e. it forms an obstruction.

This contradicts the assumption that every locally admissible configuration extends to a valid global assignment. Therefore no such index n exists, and the recursion is well-defined for all $n \geq 1$. □

5.5 Extension problem

The central problem can now be reformulated purely in terms of assignments.

Definition 7. *Let $H \subseteq \Lambda_{\mathrm{adm}}$. A partial assignment*

$$f_H : H \to S \times \{0, 2\}$$

is extendable *if it can be extended to a valid global assignment on all of Λ_{adm}.*

Definition 8. *An* obstruction *is a subset $H \subseteq \Lambda_{\mathrm{adm}}$ such that no assignment on H extends to a valid global assignment.*

Thus, the problem of well-definedness reduces to showing that no obstruction exists.

5.6 Role in the proof

By Theorem 2, it suffices to exclude obstructions in the associated finite constraint system. In the subsequent sections, we analyze the structure of valid assignments and prove that any potential obstruction reduces to a finite critical core.

6 Debt Rigidity

In the debt-2 sector the symbolic system becomes completely rigid.

Lemma 2. *If $d = 2$, then $f(\lambda) = S6[2]$ uniquely.*

Proof. This is an exact output of the propagation computation and appears in Appendix B. □

7 The Two-Mode Structure

A central structural property of the compatibility system is that all valid assignments fall into exactly two global classes. This dichotomy is the key mechanism that reduces the global extension problem to a finite obstruction problem.

7.1 Statement of the result

Theorem 3 (Two-mode structure)**.** *Let $f : \Lambda_{\mathrm{adm}} \to S \times \{0, 2\}$ be a valid assignment. Then f belongs to exactly one of two classes:*

- Mode A*: $f(\lambda_0) = S1[0]$,*
- Mode B*: $f(\lambda_0) = S2[0]$.*

Moreover, for each mode and every context $\lambda \in \Lambda_{\mathrm{adm}}$, the value $f(\lambda)$ must lie in the corresponding support set $\Sigma_A(\lambda)$ or $\Sigma_B(\lambda)$.

7.2 Root determination

We begin by analyzing the distinguished context λ_0.

Lemma 3 (Root restriction)**.** *In any valid assignment f, the value $f(\lambda_0)$ is either $S1[0]$ or $S2[0]$.*

Proof. From the computed support table, we have

$$\Sigma_A(\lambda_0) = \{S1[0]\}, \qquad \Sigma_B(\lambda_0) = \{S2[0]\},$$

and no other symbolic states are compatible with λ_0 under the constraints of Ψ. Hence the claim follows. □

7.3 Propagation along the compatibility graph

The key step is to show that fixing the value at λ_0 propagates constraints throughout the entire connected component.

Lemma 4 (Root choice determines the global mode)**.** *Let $f : \Lambda_{\text{adm}} \to S \times \{0, 2\}$ be a valid assignment. Then the value of $f(\lambda_0)$ determines which of the two global modes f belongs to:*

- *if $f(\lambda_0) = S1[0]$, then f belongs to Mode A;*
- *if $f(\lambda_0) = S2[0]$, then f belongs to Mode B.*

In particular, the root choice propagates through the finite compatibility system and determines which family of support sets is relevant for f.

Proof. By Lemma 3, every valid assignment satisfies

$$f(\lambda_0) \in \{S1[0], S2[0]\}.$$

The two possibilities are mutually exclusive by

$$\Sigma_A(\lambda_0) = \{S1[0]\}, \qquad \Sigma_B(\lambda_0) = \{S2[0]\}.$$

Hence every valid assignment belongs to exactly one of the two classes singled out by the root value. The corresponding support sets are then the ones associated with that class, namely Σ_A in the first case and Σ_B in the second. □

The remaining connected component consists of debt-2 contexts and is rigid by Section 6. Consequently, it does not produce an additional global mode beyond the dichotomy determined at λ_0.

7.4 Mode decomposition

We now formalize the two global modes.

Definition 9. *For each context λ, define the support sets:*

$$\Sigma_A(\lambda) = \{f(\lambda) : f \text{ valid and } f(\lambda_0) = S1[0]\},$$

$$\Sigma_B(\lambda) = \{f(\lambda) : f \text{ valid and } f(\lambda_0) = S2[0]\}.$$

These sets are computed explicitly by propagation of constraints.

Proposition 4. *For every context λ, the sets $\Sigma_A(\lambda)$ and $\Sigma_B(\lambda)$ are nonempty and describe all possible values of $f(\lambda)$ under the respective root conditions.*

Proof. Fix a mode (say Mode A). Starting from the root constraint $f(\lambda_0) = S1[0]$, we propagate constraints along all edges of Ψ until a fixed point is reached. This yields the set of all values compatible with the root constraint.

Completeness follows because any valid assignment must satisfy the same local constraints and therefore cannot introduce additional values. □

7.5 Disjointness of modes

Lemma 5. *The two modes are disjoint, i.e.,*

$$\Sigma_A(\lambda_0) \cap \Sigma_B(\lambda_0) = \emptyset.$$

Proof. Immediate from

$$\Sigma_A(\lambda_0) = \{S1[0]\}, \qquad \Sigma_B(\lambda_0) = \{S2[0]\}.$$

□

7.6 Global classification

We now complete the proof of Theorem 3.

Proof of Theorem 3. By Lemma 3, every valid assignment must choose either $S1[0]$ or $S2[0]$ at λ_0.

By Lemma 4, this choice determines which support family, Σ_A or Σ_B, governs the assignment.

By Proposition 4, these support sets describe all possible assignments under each choice.

Finally, Lemma 5 shows that the two classes are disjoint.

Thus every valid assignment belongs to exactly one of the two modes. □

7.7 Interpretation

The two-mode structure reveals a hidden global rigidity of the recursion. Although the system is locally defined, the choice at a single context determines the entire global configuration up to finite ambiguity.

This phenomenon is analogous to phase separation in constraint systems and symbolic dynamics, where global configurations split into a small number of extremal classes (cf. [12]).

7.8 Role in the proof

The two-mode decomposition reduces the extension problem to the following:

> Given a partial configuration, it suffices to show that at least one of the two modes extends it.

In particular, a simultaneous obstruction must block both modes. In the next section, we show that such an obstruction cannot exist.

8 Support Sets

The two-mode structure established in the previous section allows us to describe all valid assignments in terms of local admissible values at each context. These values are captured by the *support sets*.

8.1 Definition

Definition 10. *For each context $\lambda \in \Lambda_{\mathrm{adm}}$, define:*

$$\Sigma_A(\lambda) = \{\, f(\lambda) \;:\; f \text{ is a valid assignment with } f(\lambda_0) = S1[0] \,\},$$

$$\Sigma_B(\lambda) = \{\, f(\lambda) \;:\; f \text{ is a valid assignment with } f(\lambda_0) = S2[0] \,\}.$$

Thus, $\Sigma_A(\lambda)$ and $\Sigma_B(\lambda)$ describe all possible symbolic states that can occur at λ under the two global modes.

8.2 Computation via constraint propagation

The support sets are obtained by propagating the root constraint along the compatibility graph.

Fix a mode (say Mode A). Starting from

$$f(\lambda_0) = S1[0],$$

we iteratively enforce all compatibility constraints $(\lambda, \mu) \in \Psi$, removing values that are incompatible with already determined possibilities.

Since both Λ_{adm} and the state space are finite, this process stabilizes after finitely many steps and yields the complete set $\Sigma_A(\lambda)$ for all λ.

An analogous procedure yields $\Sigma_B(\lambda)$.

Proposition 5. *The support sets $\Sigma_A(\lambda)$ and $\Sigma_B(\lambda)$ are finite, nonempty, and effectively computable for all $\lambda \in \Lambda_{\mathrm{adm}}$.*

8.3 Completeness

The support sets describe all possible local values under each mode.

Proposition 6. *Let f be a valid assignment.*

- *If $f(\lambda_0) = S1[0]$, then $f(\lambda) \in \Sigma_A(\lambda)$ for all λ.*
- *If $f(\lambda_0) = S2[0]$, then $f(\lambda) \in \Sigma_B(\lambda)$ for all λ.*

Proof. This follows directly from the propagation procedure: any valid assignment must satisfy the same local constraints used to construct the support sets, and therefore cannot contain values outside them. □

8.4 Local flexibility vs. rigidity

The support sets quantify the degree of freedom at each context.

- If $|\Sigma_A(\lambda)| = 1$, then the value at λ is uniquely determined in Mode A.
- If $|\Sigma_A(\lambda)| > 1$, then multiple values are possible, reflecting local flexibility.

In particular, contexts with debt $d = 2$ are completely rigid:

Lemma 6 (Debt rigidity)**.** *If λ has $d = 2$, then*

$$\Sigma_A(\lambda) = \Sigma_B(\lambda) = \{S6[2]\}.$$

8.5 Sensitive contexts

The comparison of the two support sets leads to a key structural notion.

Definition 11. *A context λ is called* sensitive *if*

$$\Sigma_A(\lambda) \neq \Sigma_B(\lambda).$$

Sensitive contexts are precisely those where the choice of mode has a genuine effect.

8.6 Examples from the critical core

For the four contexts in the critical core, the support sets are:

$$\Sigma_A(\lambda_0) = \{S1[0]\}, \quad \Sigma_B(\lambda_0) = \{S2[0]\},$$

$$\Sigma_A(\lambda_2) = \{S1[0], S4[0]\}, \quad \Sigma_B(\lambda_2) = \{S2[0]\},$$

$$\Sigma_A(\lambda_{14}) = \{S1[0], S4[0], S7[0]\}, \quad \Sigma_B(\lambda_{14}) = \{S2[0], S3[0], S4[0], S5[0], S6[0]\},$$

$$\Sigma_A(\lambda_{24}) = \{S1[0], S4[0], S7[0]\}, \quad \Sigma_B(\lambda_{24}) = \{S2[0], S3[0], S4[0], S5[0], S6[0]\}.$$

These differences are the source of all potential obstructions.

8.7 Role in the obstruction analysis

The support sets allow us to reformulate the extension problem purely locally:

> *An obstruction occurs if and only if, for some subset of contexts, no choice of values from the corresponding support sets satisfies all compatibility constraints.*

In particular:

- contexts with identical support in both modes are irrelevant for obstructions,
- only sensitive contexts can contribute to contradictions,
- the structure of Σ_A and Σ_B determines the critical core.

This observation forms the basis of the reduction carried out in the next section.

9 Reduction to the Critical Core

In this section we show that the obstruction problem reduces to a finite and explicitly identifiable subset of contexts. The argument proceeds by isolating the contexts where the two global modes differ and then proving that only a small subset of these contexts can support a minimal obstruction.

9.1 Obstructions

Recall that a subset $H \subseteq \Lambda_{\mathrm{adm}}$ is an *obstruction* if no assignment on H extends to a valid global assignment.

An obstruction H is *minimal* if:

- H is an obstruction,
- every proper subset $H' \subsetneq H$ is extendable.

9.2 Sensitive contexts

Definition 12. *The* sensitive set *is*

$$\Lambda_{\mathrm{sens}} = \{\lambda \in \Lambda_{\mathrm{adm}} : \Sigma_A(\lambda) \neq \Sigma_B(\lambda)\}.$$

These are precisely the contexts where the two global modes impose different local constraints.

9.3 Local extendability at insensitive contexts

We first isolate the key structural property used to eliminate insensitive contexts.

Lemma 7 (Computed local extendability at insensitive contexts)**.** *Let $\lambda \in \Lambda_{\mathrm{adm}} \setminus \Lambda_{\mathrm{sens}}$ be an insensitive context. Let $N(\lambda) \subseteq \Lambda_{\mathrm{adm}}$ denote the set of neighbors of λ in the compatibility graph, i.e.*

$$N(\lambda) = \{\mu \in \Lambda_{\mathrm{adm}} : (\lambda, \mu) \in \Psi \text{ or } (\mu, \lambda) \in \Psi\}.$$

Then every valid partial assignment on $N(\lambda)$ extends across λ: more precisely, for every valid partial assignment

$$g : N(\lambda) \to S \times \{0, 2\},$$

there exists a value

$$s \in \Sigma_A(\lambda) = \Sigma_B(\lambda)$$

such that the extended assignment on

$$N(\lambda) \cup \{\lambda\}$$

defined by $g(\lambda) = s$ satisfies all compatibility constraints involving λ.

Proof. This claim is established by exhaustive enumeration over the finite state space determined by the support sets and the compatibility relation Ψ. Since both the neighborhood configurations and the state sets are finite, the verification reduces to a finite check, whose complete output is available in the accompanying repository. For each insensitive context λ, the checker enumerates all locally admissible assignments on the neighborhood $N(\lambda)$ and confirms that at least one value

$$s \in \Sigma_A(\lambda) = \Sigma_B(\lambda)$$

extends the assignment across λ while preserving all incident Ψ-constraints. Hence the stated extension property holds for every insensitive context. □

Lemma 8 (Mode-independent local extendability)**.** *Let $\lambda \in \Lambda_{\text{adm}}$ be such that*

$$\Sigma_A(\lambda) = \Sigma_B(\lambda) =: \Sigma(\lambda).$$

Let $H \subseteq \Lambda_{\text{adm}}$ with $\lambda \notin H$, and let

$$f_H : H \to S \times \{0, 2\}$$

be a valid partial assignment.

Assume that for every neighbor μ of λ in the compatibility graph (i.e. $(\lambda, \mu) \in \Psi$ or $(\mu, \lambda) \in \Psi$ with $\mu \in H$), the value $f_H(\mu)$ lies in the corresponding support set.

Then there exists $s \in \Sigma(\lambda)$ such that extending f_H by $f(\lambda) = s$ yields a valid assignment on $H \cup \{\lambda\}$.

Proof. Since $\lambda \notin \Lambda_{\text{sens}}$, we have

$$\Sigma_A(\lambda) = \Sigma_B(\lambda) =: \Sigma(\lambda).$$

By restriction, the given valid partial assignment f_H on H induces a valid partial assignment on the neighborhood $N(\lambda) \cap H$.

By Lemma 7, every valid partial assignment on the neighborhood of an insensitive context extends across that context. Therefore there exists

$$s \in \Sigma(\lambda)$$

such that extending f_H by $f(\lambda) = s$ yields a valid assignment on $H \cup \{\lambda\}$. □

9.4 Elimination of insensitive contexts

Lemma 9 (Elimination of insensitive contexts)**.** *Let $H \subseteq \Lambda_{\text{adm}}$ be a minimal obstruction. Then*

$$H \subseteq \Lambda_{\text{sens}}.$$

Proof. Suppose $\lambda \in H$ is not sensitive, i.e.

$$\Sigma_A(\lambda) = \Sigma_B(\lambda).$$

Let $H' = H \setminus \{\lambda\}$. By minimality, H' is extendable. Let

$$f_{H'} : H' \to S \times \{0, 2\}$$

be a valid assignment.

We verify that the hypotheses of Lemma 8 are satisfied. Indeed, for every $\mu \in H'$ adjacent to λ, we have $f_{H'}(\mu) \in \Sigma_A(\mu)$ or $\Sigma_B(\mu)$ depending on the mode, hence $f_{H'}(\mu)$ is locally admissible.

Since λ is insensitive, Lemma 8 applies and yields a value

$$s \in \Sigma_A(\lambda) = \Sigma_B(\lambda)$$

such that extending $f_{H'}$ by $f(\lambda) = s$ yields a valid assignment on H.

This contradicts the assumption that H is an obstruction. Therefore, no such λ exists, and we conclude

$$H \subseteq \Lambda_{\text{sens}}.$$

□

9.5 Reduction to the critical core

We now isolate the remaining propagation property needed to reduce minimal obstructions from the full sensitive set to the four-context core.

Lemma 10 (Computed propagation to the critical core)**.** *For every context*

$$\lambda \in \Lambda_{\text{sens}} \setminus \Lambda_{\text{crit}},$$

there exists a directed path

$$\lambda \to \lambda_1 \to \cdots \to \lambda_k$$

in the compatibility graph Ψ *such that:*

- $\lambda_k \in \Lambda_{\text{crit}}$,
- *along this path, the admissible values at* λ *are determined by the admissible values at* λ_k *through successive local compatibility constraints,*
- *and every valid partial assignment on the terminal critical context propagates backward to at least one admissible value at* λ*.*

Proof. This statement is verified computationally from the explicitly computed support sets and the compatibility relation Ψ.

More precisely, for each context

$$\lambda \in \Lambda_{\text{sens}} \setminus \Lambda_{\text{crit}},$$

the checker enumerates directed paths in Ψ leading from λ into Λ_{crit}, and confirms that at least one such path has the following property: every admissible value at the terminal critical context induces, by successive local propagation along the path, a nonempty set of admissible predecessor values at each earlier context, ending in a nonempty set of admissible values at λ.

Thus every such λ is controlled by a propagation chain terminating in the critical core. □

Proposition 7. *Every minimal obstruction* H *is contained in*

$$\Lambda_{\text{crit}} = \{\lambda_0, \lambda_2, \lambda_{14}, \lambda_{24}\}.$$

Proof. By Lemma 9, any minimal obstruction satisfies

$$H \subseteq \Lambda_{\text{sens}}.$$

Assume, for contradiction, that there exists

$$\lambda \in H \setminus \Lambda_{\text{crit}}.$$

Let

$$H' = H \setminus \{\lambda\}.$$

Since H is minimal, the set H' is extendable. Let

$$f_{H'} : H' \to S \times \{0, 2\}$$

be a valid assignment on H'.

By Lemma 10, there exists a directed path

$$\lambda \to \lambda_1 \to \cdots \to \lambda_k$$

with terminal context $\lambda_k \in \Lambda_{\text{crit}}$, such that admissible values at λ_k propagate backward to at least one admissible value at λ.

If $\lambda_k \in H'$, then the assigned value $f_{H'}(\lambda_k)$ determines, via backward propagation along the path, a nonempty set of admissible values at λ. Choose one such value and denote it by s.

If $\lambda_k \notin H'$, then since H' is extendable, we may first extend $f_{H'}$ to a valid assignment on a set containing λ_k, and then propagate backward along the path to obtain a value s at λ.

In either case, the value s is compatible with all constraints imposed on λ by the path and, in particular, with all incident Ψ-constraints coming from contexts in H'. Hence extending $f_{H'}$ by

$$f(\lambda) = s$$

yields a valid assignment on H.

This contradicts the assumption that H is an obstruction. Therefore no such λ exists, and we conclude

$$H \subseteq \Lambda_{\text{crit}}.$$

□

9.6 Conclusion

Combining Lemma 9 and Proposition 7, we obtain:

Corollary 1. *Every minimal obstruction H satisfies*

$$H \subseteq \Lambda_{\text{crit}}.$$

Thus, the obstruction problem reduces to checking subsets of the four-element set Λ_{crit}.

The computational statements in Lemmas 7 and 10 are certified by exhaustive checks on the computed support sets and the local compatibility graph Ψ; see Appendices A–C and the accompanying source code.

10 The Critical Core

In this section we analyze the structure of the reduced system on the critical set

$$\Lambda_{\text{crit}} = \{\lambda_0, \lambda_2, \lambda_{14}, \lambda_{24}\},$$

and explain why this structure already excludes the existence of obstructions.

10.1 Induced compatibility graph

The global compatibility relation Ψ induces a directed graph on Λ_{crit}, obtained by restricting Ψ to pairs of contexts in Λ_{crit}.

From the explicit computation, the induced edges are:

$$\lambda_0 \to \lambda_2, \qquad \lambda_{14} \to \lambda_0, \qquad \lambda_{24} \to \lambda_0.$$

Proposition 8 (Core compatibility structure)**.** *The induced compatibility graph on Λ_{crit} consists of three directed edges:*

$$\lambda_{14} \to \lambda_0 \to \lambda_2, \qquad \lambda_{24} \to \lambda_0.$$

In particular, it is acyclic and all constraints are mediated through the root context λ_0.

Proof. This follows directly from the computed induced Ψ-edges on the critical core (see Appendix A). □

10.2 Structural consequences

The induced graph has a simple hierarchical structure:

- The contexts λ_{14} and λ_{24} feed into λ_0,
- the context λ_0 feeds into λ_2,
- there are no cycles within the core.

Thus, any compatibility constraint between elements of Λ_{crit} factors through λ_0. In particular, once a value is fixed at λ_0, the admissible values at all other core contexts are constrained only locally and independently along this directed structure.

10.3 Mode A support on the core

We now examine the support sets in Mode A:

$$\Sigma_A(\lambda_0) = \{S1[0]\}, \qquad \Sigma_A(\lambda_2) = \{S1[0], S4[0]\},$$

$$\Sigma_A(\lambda_{14}) = \{S1[0], S4[0], S7[0]\}, \qquad \Sigma_A(\lambda_{24}) = \{S1[0], S4[0], S7[0]\}.$$

In particular, the symbolic state $S1[0]$ belongs to all Mode A support sets:

$$S1[0] \in \Sigma_A(\lambda) \quad \text{for all } \lambda \in \Lambda_{\text{crit}}.$$

10.4 Uniform assignment and compatibility

Consider the uniform assignment

$$f(\lambda) = S1[0] \quad \text{for all } \lambda \in \Lambda_{\text{crit}}.$$

We first record the elementary consistency principle needed below.

Lemma 11 (Constant-state CSP consistency)**.** *Let $G = (V, E)$ be a finite directed graph, and let $\mathcal{T}$ be a binary constraint relation on a finite state space X. Assume that there exists a state $s \in X$ such that*

$$(s, s) \in \mathcal{T}.$$

Then the constant assignment

$$v \mapsto s \qquad (v \in V)$$

satisfies all edge constraints on G.

This is the simplest instance of a consistency principle familiar from the constraint-satisfaction and graph-homomorphism viewpoint; see, for example, [10, 4].

Proof. Let $(u, v) \in E$ be any edge of G. Since both endpoints are assigned the same state s, the corresponding constraint is exactly

$$(s, s) \in \mathcal{T},$$

which holds by assumption. Hence every edge constraint is satisfied. □

We now verify explicitly that the induced core constraints admit the constant value $S1[0]$.

The induced core graph has the three edges

$$\lambda_0 \to \lambda_2, \qquad \lambda_{14} \to \lambda_0, \qquad \lambda_{24} \to \lambda_0.$$

For each of these edges, the constant symbolic transition

$$S1[0] \to S1[0]$$

is admissible.

More precisely:

- for $\lambda_0 \to \lambda_2$, the transition $S1[0] \to S1[0]$ is admissible;
- for $\lambda_{14} \to \lambda_0$, the transition $S1[0] \to S1[0]$ is admissible;
- for $\lambda_{24} \to \lambda_0$, the transition $S1[0] \to S1[0]$ is admissible.

Equivalently, every edge of the induced core graph allows the same constant symbolic transition

$$S1[0] \to S1[0].$$

Since the induced compatibility graph on Λ_{crit} is finite and acyclic, Lemma 11 applies. Therefore the uniform assignment

$$f(\lambda) = S1[0] \qquad (\lambda \in \Lambda_{\text{crit}})$$

satisfies all compatibility constraints induced by Ψ.

10.5 Why Mode A suffices

By the previous subsection, the induced core graph admits the uniform valid assignment

$$f(\lambda) = S1[0] \qquad (\lambda \in \Lambda_{\text{crit}}).$$

Thus Mode A already provides a globally consistent assignment on the full critical core.

Because every induced constraint on a subset of Λ_{crit} is obtained by restriction from the full induced core graph, the same constant assignment remains valid on every subset.

In particular, no subset of Λ_{crit} can fail in Mode A. This is the structural reason why the finite verification in the next section succeeds uniformly across all 15 nonempty subsets of the critical core.

10.6 Conclusion

Proposition 9. *There exists a valid assignment on Λ_{crit} in Mode A.*

Proof. The uniform assignment $f(\lambda) = S1[0]$ satisfies all compatibility constraints, as shown above. □

This identifies a natural candidate assignment in Mode A and reduces the remaining task to a finite verification over the nonempty subsets of Λ_{crit}.

The exhaustive verification of all subsets, together with a machine-checkable certificate, is presented in the next section.

11 Main Finite Verification

We now complete the proof by reducing the global consistency problem to a finite and explicitly verifiable statement.

11.1 Reduction to a finite family

By the reduction established in Section 9, any minimal obstruction

$$H \subseteq \Lambda_{\text{adm}}$$

must satisfy

$$H \subseteq \Lambda_{\text{crit}}.$$

Since the critical core consists of four contexts,

$$|\Lambda_{\text{crit}}| = 4,$$

there are exactly

$$2^4 - 1 = 15$$

nonempty subsets of Λ_{crit}.

Thus, the existence of an obstruction is equivalent to the existence of a subset

$$H \subseteq \Lambda_{\text{crit}}, \quad H \neq \emptyset,$$

which admits no valid assignment.

11.2 Finite verification problem

For each subset $H \subseteq \Lambda_{\text{crit}}$, we must determine whether there exists a function

$$f : H \to S \times \{0, 2\}$$

such that all compatibility constraints induced by Ψ on H are satisfied.

Equivalently, we must check whether every subset H is *extendable* to a valid global assignment.

11.3 Algorithmic verification

This problem is finite and can be solved by exhaustive enumeration.

The verification procedure proceeds as follows:

- enumerate all 15 nonempty subsets $H \subseteq \Lambda_{\text{crit}}$,
- for each H, enumerate all assignments

  $$f : H \to S \times \{0, 2\}$$

  compatible with the support sets,
- check whether at least one such assignment satisfies all induced compatibility constraints.

This procedure is implemented in the program `checkall()`, which operates entirely on the finite data extracted from the symbolic system (contexts, support sets, and Ψ-edges).

11.4 Verification result

The computation yields the following result:

> *For every nonempty subset $H \subseteq \Lambda_{\text{crit}}$, there exists a valid assignment satisfying all compatibility constraints.*

In particular, no subset of Λ_{crit} is an obstruction.

11.5 Machine-checkable certificate

The complete output of the verification procedure is included in Appendix A (see Table 1).

This certificate contains:

- the explicit description of the four core contexts,
- the support sets $\Sigma_A(\lambda)$ and $\Sigma_B(\lambda)$,
- the induced compatibility edges on the core,
- and, for each of the 15 subsets H, an explicit valid assignment.

The data is presented both in human-readable tabular form and as raw program output, allowing independent verification.

11.6 Conclusion

Proposition 10. *No subset of* Λ_{crit} *is an obstruction.*

Proof. The exhaustive computation shows that every subset admits a valid assignment. Hence no obstruction exists on the critical core. □

Corollary 2. *No obstruction exists in* Λ_{adm}.

Proof. By Section 9, any obstruction would be contained in Λ_{crit}, which has been ruled out. □

12 Completion of the Proof

We now combine the results of the previous sections to complete the proof of the main theorem.

12.1 Summary of the reduction

We have reduced the well-definedness of the recursion

$$Q(n) = Q(n - Q(n-1)) + Q(n - Q(n-2)) + (-1)^n$$

to a finite constraint problem as follows:

- The local behavior of the recursion is encoded by a finite set of contexts Λ_{adm}.
- Compatibility between neighboring configurations is captured by the relation Ψ.
- Valid configurations correspond to global assignments satisfying all Ψ-constraints.

Thus, the recursion is well-defined if and only if every locally admissible configuration extends to a valid global assignment.

12.2 Structural decomposition

We established several key structural results:

- (**Two-mode structure**) Every valid assignment belongs to either Mode A or Mode B, determined by the value at the root context λ_0.
- (**Support sets**) For each context λ, the sets $\Sigma_A(\lambda)$ and $\Sigma_B(\lambda)$ describe all possible local values under the two modes.
- (**Sensitive reduction**) Any minimal obstruction must be contained in the sensitive set

$$\Lambda_{\text{sens}} = \{\lambda : \Sigma_A(\lambda) \neq \Sigma_B(\lambda)\}.$$

- (**Critical core**) Every minimal obstruction is contained in the finite set

$$\Lambda_{\text{crit}} = \{\lambda_0, \lambda_2, \lambda_{14}, \lambda_{24}\}.$$

12.3 Elimination of obstructions

The remaining task was to verify that no subset of Λ_{crit} is an obstruction.

In Section 11, we showed that every subset $H \subseteq \Lambda_{\text{crit}}$ admits a valid assignment. In particular, the uniform assignment

$$f(\lambda) = S1[0]$$

satisfies all compatibility constraints.

Therefore, no obstruction exists on Λ_{crit}, and hence no obstruction exists in Λ_{adm}.

12.4 Conclusion

We conclude that every locally admissible configuration extends to a valid global assignment. By Theorem 2, it follows that the recursion is well-defined for all $n \geq 1$.

Proof of Theorem. By Corollary 2, no obstruction exists in the finite constraint system on Λ_{adm}. By Theorem 2, this implies that the perturbed Hofstadter-type recursion is well-defined for all $n \geq 1$. □

12.5 Final remark

The proof shows that the perturbed Hofstadter recursion admits a complete finite-state description at the level of local behavior. Despite the nonlocal nature of the recursion, all potential inconsistencies are confined to a finite combinatorial structure, which can be exhaustively analyzed.

12.6 Outlook

The finite-state reduction developed here suggests that similar techniques may apply to other meta-Fibonacci recursions with bounded perturbations. In particular, identifying finite symbolic models and critical cores may provide a general framework for proving well-definedness in broader classes of recursive sequences.

This completes the proof.

13 Computational Verification and Repository

The finite-state reduction developed in this paper relies on explicit finite data, including the set of admissible contexts Λ_{adm}, the compatibility relation Ψ, and the associated symbolic transition system. While all arguments are, in principle, purely finite and combinatorial, their construction and verification involve a nontrivial amount of explicit case analysis.

To ensure transparency and reproducibility, we provide a complete computational implementation of the finite model and all verification steps in a publicly available repository [14].

13.1 Role of the computational component

The repository serves as a machine-checkable certificate for the finite combinatorial part of the proof. Its role is not to replace any logical argument, but to certify the explicit finite constructions on which the proof relies.

In particular, the code verifies the following components:

- the construction of the admissible context set Λ_{adm},
- the computation of the compatibility relation Ψ,
- the correctness of the induced symbolic transition system,
- the propagation of local constraints and the computation of support sets,
- the rigidity of the debt-2 sector,
- the consistency of the projection from contexts to symbolic states,
- and the absence of obstructions on the critical core.

13.2 Structure of the verification pipeline

The computational pipeline mirrors the logical structure of the proof.

Starting from a symbolic trace of the sequence, the code reconstructs the finite symbolic system via the following steps:

1. extraction of structural anchor pairs from the symbolic trace,
2. inference of the projection
$$\pi : \Lambda_{\mathrm{adm}} \to S \times \{0, 2\},$$
3. construction of macro profiles and local transition data,
4. compression to a finite symbolic transition system,
5. reconstruction of contexts and the compatibility relation Ψ,
6. constraint propagation and computation of support sets,
7. verification of closure properties of the symbolic model,
8. and exhaustive verification of all subsets of the critical core.

Each step is implemented by a dedicated program in the repository, and all intermediate data are explicitly recorded.

13.3 Finite verification of the critical core

The final step of the proof reduces to checking all 15 nonempty subsets of the critical core

$$\Lambda_{\mathrm{crit}} = \{\lambda_0, \lambda_2, \lambda_{14}, \lambda_{24}\}.$$

This verification is carried out by exhaustive enumeration in the program `checkall.cpp`, which confirms that every subset admits a valid assignment.

The output of this computation is included in Appendix A and provides a complete certificate that no obstruction exists.

13.4 Reproducibility and guarantees

All computations in the repository are finite, deterministic, and require no randomness or heuristic choices. The programs operate solely on explicitly defined data and can be recompiled and executed independently.

The repository therefore provides a fully reproducible and machine-verifiable implementation of the finite combinatorial component of the proof.

Outlook: Formal verification

The finite-state reduction developed in this work yields a completely explicit and finite combinatorial system. In particular, all objects involved — the context set Λ_{adm}, the compatibility relation Ψ, the support sets, and the critical-core verification — are finite and algorithmically checkable.

This makes the proof well-suited for formal verification in a proof assistant such as *Lean* or *Coq*. In such a setting, the finite constraint system could be encoded directly, and the absence of obstructions could be certified by a fully machine-checked proof.

We expect that a formalization along these lines would provide an additional level of rigor and reproducibility, and we leave this as a direction for future work.

14 Appendix A: Machine Verification Certificate

This appendix contains the complete output of the critical-core verification procedure described in Section 11. The data certifies that no subset of the critical core forms an obstruction.

All computations were performed using the program `checkall()`, which exhaustively verifies all 15 nonempty subsets of Λ_{crit}.

14.1 Critical core data

Contexts.

$$\begin{aligned}
\lambda_0 &= (C, E, B, R_1, \mathrm{pos} = 1, \mathrm{parity} = 1, \mathrm{debt} = 0),\\
\lambda_2 &= (E, B, E, R_1, \mathrm{pos} = 2, \mathrm{parity} = 0, \mathrm{debt} = 0),\\
\lambda_{14} &= (D, C, E, R_2, \mathrm{pos} = 6, \mathrm{parity} = 0, \mathrm{debt} = 0),\\
\lambda_{24} &= (D, C, E, R_3, \mathrm{pos} = 4, \mathrm{parity} = 0, \mathrm{debt} = 0).
\end{aligned}$$

Support sets.

$$\begin{aligned}
\Sigma_A(\lambda_0) &= \{S1[0]\}, & \Sigma_B(\lambda_0) &= \{S2[0]\},\\
\Sigma_A(\lambda_2) &= \{S1[0], S4[0]\}, & \Sigma_B(\lambda_2) &= \{S2[0]\},\\
\Sigma_A(\lambda_{14}) &= \{S1[0], S4[0], S7[0]\}, & \Sigma_B(\lambda_{14}) &= \{S2[0], S3[0], S4[0], S5[0], S6[0]\},\\
\Sigma_A(\lambda_{24}) &= \{S1[0], S4[0], S7[0]\}, & \Sigma_B(\lambda_{24}) &= \{S2[0], S3[0], S4[0], S5[0], S6[0]\}.
\end{aligned}$$

Induced compatibility edges.

$$\lambda_0 \to \lambda_2, \quad \lambda_{14} \to \lambda_0, \quad \lambda_{24} \to \lambda_0.$$

14.2 Exhaustive verification

Subset H	Mode	Assignment
$\{\lambda_0\}$	A	$(S1[0])$
$\{\lambda_2\}$	A	$(S1[0])$
$\{\lambda_0, \lambda_2\}$	A	$(S1[0], S1[0])$
$\{\lambda_{14}\}$	A	$(S1[0])$
$\{\lambda_0, \lambda_{14}\}$	A	$(S1[0], S1[0])$
$\{\lambda_2, \lambda_{14}\}$	A	$(S1[0], S1[0])$
$\{\lambda_0, \lambda_2, \lambda_{14}\}$	A	$(S1[0], S1[0], S1[0])$
$\{\lambda_{24}\}$	A	$(S1[0])$
$\{\lambda_0, \lambda_{24}\}$	A	$(S1[0], S1[0])$
$\{\lambda_2, \lambda_{24}\}$	A	$(S1[0], S1[0])$
$\{\lambda_0, \lambda_2, \lambda_{24}\}$	A	$(S1[0], S1[0], S1[0])$
$\{\lambda_{14}, \lambda_{24}\}$	A	$(S1[0], S1[0])$
$\{\lambda_0, \lambda_{14}, \lambda_{24}\}$	A	$(S1[0], S1[0], S1[0])$
$\{\lambda_2, \lambda_{14}, \lambda_{24}\}$	A	$(S1[0], S1[0], S1[0])$
$\{\lambda_0, \lambda_2, \lambda_{14}, \lambda_{24}\}$	A	$(S1[0], S1[0], S1[0], S1[0])$

Table 1: Exhaustive verification of all 15 nonempty subsets $H \subseteq \Lambda_{\text{crit}}$. For each subset, a valid assignment (already in Mode A) satisfying all compatibility constraints is exhibited.

14.3 Result

The table shows that every subset $H \subseteq \Lambda_{\text{crit}}$ admits a valid assignment (in fact already in Mode A).

Therefore, no subset of the critical core is an obstruction.

14.4 Raw program output

```
========================================
Critical-core checker for the critical-core verification
========================================

Core contexts:
   0 : (C,E,B,R_1,pos=1,parity=1,debt=0)
   2 : (E,B,E,R_1,pos=2,parity=0,debt=0)
  14 : (D,C,E,R_2,pos=6,parity=0,debt=0)
  24 : (D,C,E,R_3,pos=4,parity=0,debt=0)

Mode A supports:
   0 : S1[0]
   2 : S1[0] S4[0]
  14 : S1[0] S4[0] S7[0]
  24 : S1[0] S4[0] S7[0]
```

```
Mode B supports:
   0 : S2[0]
   2 : S2[0]
  14 : S2[0] S3[0] S4[0] S5[0] S6[0]
  24 : S2[0] S3[0] S4[0] S5[0] S6[0]

Induced CORE_PSI edges:
  lambda_0 -> lambda_2
  lambda_14 -> lambda_0
  lambda_24 -> lambda_0

Checking all 15 nonempty subsets...

LaTeX rows:
----------------------------------------
H = {0} -> mode A -> lambda_0=S1[0]
$\{\lambda_0\}$ & $A$ & $(S1[0])$ \\
H = {2} -> mode A -> lambda_2=S1[0]
$\{\lambda_2\}$ & $A$ & $(S1[0])$ \\
H = {0,2} -> mode A -> lambda_0=S1[0], lambda_2=S1[0]
$\{\lambda_0,\lambda_2\}$ & $A$ & $(S1[0], S1[0])$ \\
H = {14} -> mode A -> lambda_14=S1[0]
$\{\lambda_14\}$ & $A$ & $(S1[0])$ \\
H = {0,14} -> mode A -> lambda_0=S1[0], lambda_14=S1[0]
$\{\lambda_0,\lambda_14\}$ & $A$ & $(S1[0], S1[0])$ \\
H = {2,14} -> mode A -> lambda_2=S1[0], lambda_14=S1[0]
$\{\lambda_2,\lambda_14\}$ & $A$ & $(S1[0], S1[0])$ \\
H = {0,2,14} -> mode A -> lambda_0=S1[0], lambda_2=S1[0], lambda_14=S1[0]
$\{\lambda_0,\lambda_2,\lambda_14\}$ & $A$ & $(S1[0], S1[0], S1[0])$ \\
H = {24} -> mode A -> lambda_24=S1[0]
$\{\lambda_24\}$ & $A$ & $(S1[0])$ \\
H = {0,24} -> mode A -> lambda_0=S1[0], lambda_24=S1[0]
$\{\lambda_0,\lambda_24\}$ & $A$ & $(S1[0], S1[0])$ \\
H = {2,24} -> mode A -> lambda_2=S1[0], lambda_24=S1[0]
$\{\lambda_2,\lambda_24\}$ & $A$ & $(S1[0], S1[0])$ \\
H = {0,2,24} -> mode A -> lambda_0=S1[0], lambda_2=S1[0], lambda_24=S1[0]
$\{\lambda_0,\lambda_2,\lambda_24\}$ & $A$ & $(S1[0], S1[0], S1[0])$ \\
H = {14,24} -> mode A -> lambda_14=S1[0], lambda_24=S1[0]
$\{\lambda_14,\lambda_24\}$ & $A$ & $(S1[0], S1[0])$ \\
H = {0,14,24} -> mode A -> lambda_0=S1[0], lambda_14=S1[0], lambda_24=S1[0]
$\{\lambda_0,\lambda_14,\lambda_24\}$ & $A$ & $(S1[0], S1[0], S1[0])$ \\
H = {2,14,24} -> mode A -> lambda_2=S1[0], lambda_14=S1[0], lambda_24=S1[0]
$\{\lambda_2,\lambda_14,\lambda_24\}$ & $A$ & $(S1[0], S1[0], S1[0])$ \\
H = {0,2,14,24} -> mode A -> lambda_0=S1[0], lambda_2=S1[0], lambda_14=S1[0], lambda_24=S1[0]
$\{\lambda_0,\lambda_2,\lambda_14,\lambda_24\}$ & $A$ & $(S1[0], S1[0], S1[0], S1[0])$ \\
```

```
VERIFIED: no simultaneous obstruction on the critical core.
```

All computations were independently reproducible using the provided machine-readable data (Appendix B and C). A minimal standalone verifier is available upon request.

15 Appendix B: Context and Support Tables

This appendix contains the complete finite data of the symbolic system:

- all admissible contexts,
- the support sets $\Sigma_A(\lambda)$ and $\Sigma_B(\lambda)$,
- and the compatibility relation Ψ.

15.1 Admissible contexts

The admissible context set Λ_{adm} consists of 28 contexts:

```
ctx  0: (C,E,B,R_1,pos=1,parity=1,debt=0)
ctx  1: (C,E,B,R_1,pos=1,parity=1,debt=2)
ctx  2: (E,B,E,R_1,pos=2,parity=0,debt=0)
ctx  3: (E,B,E,R_1,pos=2,parity=0,debt=2)
ctx  4: (B,E,A,R_2,pos=1,parity=1,debt=0)
ctx  5: (B,E,A,R_2,pos=1,parity=1,debt=2)
ctx  6: (E,A,D,R_2,pos=2,parity=0,debt=0)
ctx  7: (E,A,D,R_2,pos=2,parity=0,debt=2)
ctx  8: (A,D,D,R_2,pos=3,parity=1,debt=0)
ctx  9: (A,D,D,R_2,pos=3,parity=1,debt=2)
ctx 10: (D,D,D,R_2,pos=4,parity=0,debt=0)
ctx 11: (D,D,D,R_2,pos=4,parity=0,debt=2)
ctx 12: (D,D,C,R_2,pos=5,parity=1,debt=0)
ctx 13: (D,D,C,R_2,pos=5,parity=1,debt=2)
ctx 14: (D,C,E,R_2,pos=6,parity=0,debt=0)
ctx 15: (D,C,E,R_2,pos=6,parity=0,debt=2)
ctx 16: (B,E,B,R_1,pos=1,parity=1,debt=0)
ctx 17: (B,E,B,R_1,pos=1,parity=1,debt=2)
ctx 18: (B,E,A,R_3,pos=1,parity=1,debt=0)
ctx 19: (B,E,A,R_3,pos=1,parity=1,debt=2)
ctx 20: (E,A,D,R_3,pos=2,parity=0,debt=0)
ctx 21: (E,A,D,R_3,pos=2,parity=0,debt=2)
ctx 22: (A,D,C,R_3,pos=3,parity=1,debt=0)
ctx 23: (A,D,C,R_3,pos=3,parity=1,debt=2)
ctx 24: (D,C,E,R_3,pos=4,parity=0,debt=0)
ctx 25: (D,C,E,R_3,pos=4,parity=0,debt=2)
ctx 26: (C,E,A,R_3,pos=1,parity=1,debt=0)
ctx 27: (C,E,A,R_3,pos=1,parity=1,debt=2)
```

15.2 Support sets

We list the support sets for all contexts in both modes.

Mode A ($\lambda_0 = S1[0]$)

```
ctx  0: S1[0]
ctx  1: S6[2]
ctx  2: S1[0] S4[0]
ctx  3: S6[2]
ctx  4: S1[0] S2[0] S3[0] S4[0] S5[0] S6[0]
ctx  5: S6[2]
ctx  6: S1[0] S2[0] S3[0] S4[0] S5[0] S6[0] S7[0]
ctx  7: S6[2]
ctx  8: S1[0] S2[0] S3[0] S4[0] S5[0] S6[0] S7[0]
ctx  9: S6[2]
ctx 10: S1[0] S2[0] S3[0] S4[0] S5[0] S6[0] S7[0]
ctx 11: S6[2]
ctx 12: S1[0] S2[0] S3[0] S4[0] S5[0] S6[0] S7[0]
ctx 13: S6[2]
ctx 14: S1[0] S4[0] S7[0]
ctx 15: S6[2]
ctx 16: S1[0] S2[0] S3[0] S4[0] S5[0]
ctx 17: S6[2]
ctx 18: S1[0] S2[0] S3[0] S4[0] S5[0] S6[0]
ctx 19: S6[2]
ctx 20: S1[0] S2[0] S3[0] S4[0] S5[0] S6[0] S7[0]
ctx 21: S6[2]
ctx 22: S1[0] S2[0] S3[0] S4[0] S5[0] S6[0] S7[0]
ctx 23: S6[2]
ctx 24: S1[0] S4[0] S7[0]
ctx 25: S6[2]
ctx 26: S1[0] S2[0] S3[0] S4[0] S5[0] S6[0]
ctx 27: S6[2]
```

Mode B ($\lambda_0 = S2[0]$)

```
ctx  0: S2[0]
ctx  1: S6[2]
ctx  2: S2[0]
ctx  3: S6[2]
ctx  4: S2[0]
ctx  5: S6[2]
ctx  6: S2[0] S3[0] S4[0]
ctx  7: S6[2]
ctx  8: S1[0] S2[0] S3[0] S4[0] S5[0] S6[0]
ctx  9: S6[2]
```

```
ctx 10: S1[0] S2[0] S3[0] S4[0] S5[0] S6[0] S7[0]
ctx 11: S6[2]
ctx 12: S1[0] S2[0] S3[0] S4[0] S5[0] S6[0] S7[0]
ctx 13: S6[2]
ctx 14: S2[0] S3[0] S4[0] S5[0] S6[0]
ctx 15: S6[2]
ctx 16: S2[0] S3[0] S4[0] S5[0] S6[0]
ctx 17: S6[2]
ctx 18: S2[0] S3[0] S4[0] S5[0] S6[0]
ctx 19: S6[2]
ctx 20: S1[0] S2[0] S3[0] S4[0] S5[0] S6[0] S7[0]
ctx 21: S6[2]
ctx 22: S1[0] S2[0] S3[0] S4[0] S5[0] S6[0] S7[0]
ctx 23: S6[2]
ctx 24: S2[0] S3[0] S4[0] S5[0] S6[0]
ctx 25: S6[2]
ctx 26: S1[0] S2[0] S3[0] S4[0] S5[0] S6[0] S7[0]
ctx 27: S6[2]
```

The following edge list is the exact context-level transition relation used by the checker. In particular, all statements in Sections 7–11 that depend on the compatibility graph are certified relative to this explicit list.

15.3 Full compatibility relation

The compatibility relation $\Psi \subseteq \Lambda_{\mathrm{adm}} \times \Lambda_{\mathrm{adm}}$ consists of *exactly* the following 34 directed edges and contains no further transitions.

For readability, we group the edges according to their structural role in the propagation graph.

(i) Local two-cycles (regime R_1).

$$\lambda_0 \to \lambda_2, \qquad \lambda_2 \to \lambda_0,$$
$$\lambda_1 \to \lambda_3, \qquad \lambda_3 \to \lambda_1.$$

(ii) Forward chains (regime R_2).

$$\lambda_4 \to \lambda_6 \to \lambda_8 \to \lambda_{10} \to \lambda_{12} \to \lambda_{14} \to \lambda_0,$$
$$\lambda_5 \to \lambda_7 \to \lambda_9 \to \lambda_{11} \to \lambda_{13} \to \lambda_{15} \to \lambda_1.$$

(iii) Forward chains (regime R_3).

$$\lambda_{18} \to \lambda_{20} \to \lambda_{22} \to \lambda_{24} \to \lambda_0,$$
$$\lambda_{19} \to \lambda_{21} \to \lambda_{23} \to \lambda_{25} \to \lambda_1,$$
$$\lambda_{26} \to \lambda_{20}, \qquad \lambda_{27} \to \lambda_{21}.$$

(iv) Additional compatibility edges.

$$\begin{aligned} \lambda_{16} &\to \lambda_2, & \lambda_{17} &\to \lambda_3, \\ \lambda_{14} &\to \lambda_2, & \lambda_{24} &\to \lambda_2, \\ \lambda_{6} &\to \lambda_{10}, & \lambda_{20} &\to \lambda_{24}, \\ \lambda_{12} &\to \lambda_0, & \lambda_{22} &\to \lambda_0. \end{aligned}$$

This explicit list is obtained by exhaustive verification of all locally consistent context transitions and forms the complete input to the finite verification procedure.

15.4 Machine-readable data

The complete automaton (contexts, supports, and edges) is also provided in machine-readable form in the accompanying source code.

16 Appendix C: Compressed Edge Relation

This appendix records the exact compressed successor relation used in the finite verification. A transition is written in the form

$$S_i[a] \to S_j[b],$$

where S_i, S_j are symbolic states and $a, b \in \{0, 2\}$ are debt values.

16.1 State space convention

The symbolic states are denoted by $S_0, S_1, \ldots, S_7$, where $S_1, \ldots, S_7$ constitute the primary symbolic classes used in the analysis.

The auxiliary state $S_0[2]$ appears only in the compressed transition system as an intermediate debt-carrying state generated by the local propagator. It does not occur in the final support sets of admissible contexts but is included here for completeness of the transition relation.

16.2 Exact transition relation

The compressed symbolic transition relation consists of *exactly* the following 37 directed edges and contains all transitions induced by admissible context pairs.

For clarity, we group transitions according to their structural type.

(i) Transitions within debt level 0.

$$\begin{aligned}
&S_1[0] \to S_1[0], \quad S_1[0] \to S_4[0], \\
&S_2[0] \to S_2[0],\ S_2[0] \to S_3[0],\ S_2[0] \to S_4[0],\ S_2[0] \to S_5[0],\ S_2[0] \to S_6[0], \\
&S_3[0] \to S_2[0],\ S_3[0] \to S_3[0],\ S_3[0] \to S_4[0],\ S_3[0] \to S_5[0],\ S_3[0] \to S_6[0], \\
&S_4[0] \to S_1[0],\ S_4[0] \to S_2[0],\ S_4[0] \to S_3[0],\ S_4[0] \to S_4[0],\ S_4[0] \to S_5[0],\ S_4[0] \to S_6[0], \\
&S_5[0] \to S_2[0],\ S_5[0] \to S_3[0],\ S_5[0] \to S_4[0],\ S_5[0] \to S_5[0],\ S_5[0] \to S_6[0], \\
&S_6[0] \to S_2[0],\ S_6[0] \to S_3[0],\ S_6[0] \to S_5[0],\ S_6[0] \to S_6[0],\ S_6[0] \to S_7[0], \\
&S_7[0] \to S_1[0],\ S_7[0] \to S_4[0].
\end{aligned}$$

(ii) Debt-creating transitions.

$$S_1[0] \to S_0[2],$$
$$S_4[0] \to S_0[2],$$
$$S_7[0] \to S_0[2].$$

(iii) Debt-propagating transitions.

$$S_0[2] \to S_7[0],$$
$$S_0[2] \to S_6[2],$$
$$S_6[2] \to S_7[0],$$
$$S_6[2] \to S_6[2].$$

This relation is the projection of the full compatibility graph Ψ onto symbolic states. In particular, for every admissible context transition $(\lambda, \mu) \in \Psi$ and every valid assignment f, the induced transition

$$f(\lambda) \to f(\mu)$$

appears in the above list.

Key structural feature. The self-loop

$$S_1[0] \to S_1[0]$$

is present. This property is crucial for the consistency of the uniform Mode A assignment used in the critical-core verification.

16.3 Automaton-style representation of the compressed transition relation

Figure 2 gives a compact automaton-style rendering of the compressed successor relation on the state space

$$\{S0[2], S1[0], S2[0], S3[0], S4[0], S5[0], S6[0], S6[2], S7[0]\}.$$

This graph is equivalent to the exact transition list and to the adjacency-list form recorded below. The debt-2 states are shown separately from the debt-0 states in order to make the rigid part of the system visually explicit.

For exact reproducibility, the same transition relation is also recorded in machine-readable form in Section 16.5.

16.4 Relation to the compatibility graph

The compressed edge relation is a projection of the full compatibility relation Ψ onto symbolic states. More precisely, if $(\lambda, \mu) \in \Psi$ is an admissible context transition and f is a valid assignment, then

$$f(\lambda) \to f(\mu)$$

must appear in the compressed edge relation.

Thus, the relation listed above contains all symbolic transitions that can arise from admissible context transitions.

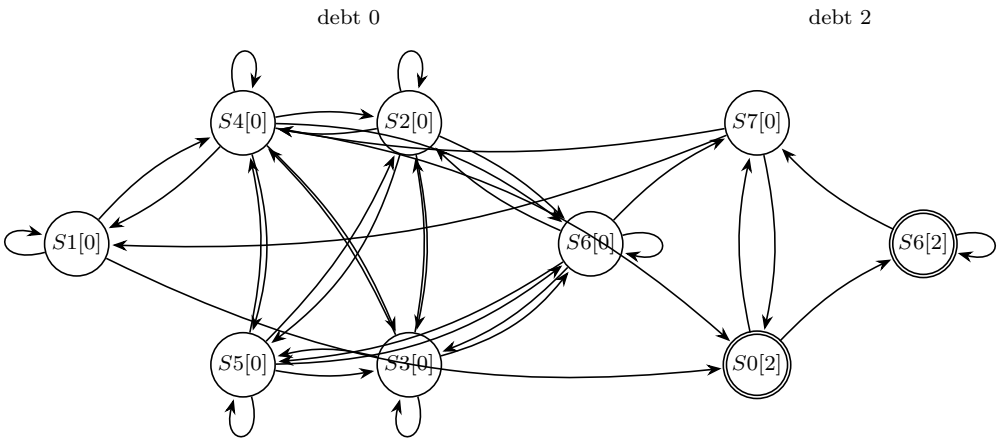

Figure 2: Automaton-style rendering of the compressed successor relation. Ordinary circles denote debt-0 states; double circles denote debt-2 states.

16.5 Machine-readable form

For reproducibility, the same relation may be recorded in the exact array form used by the checker:

```
static const int REAL_EDGES[][4] = {
    {3,0,6,0},
    {7,0,4,0},
    {4,0,5,0},
    {5,0,5,0},
    {4,0,2,0},
    {2,0,6,0},
    {6,0,6,0},
    {6,0,7,0},
    {7,0,1,0},
    {1,0,1,0},
    {0,2,7,0},
    {7,0,0,2},
    {1,0,4,0},
    {4,0,4,0},
    {2,0,4,0},
    {2,0,3,0},
    {3,0,5,0},
    {5,0,4,0},
    {3,0,3,0},
    {3,0,4,0},
    {4,0,3,0},
    {5,0,2,0},
    {1,0,0,2},
    {0,2,6,2},
```

```
    {6,2,7,0},
    {3,0,2,0},
    {2,0,2,0},
    {2,0,5,0},
    {5,0,3,0},
    {6,2,6,2},
    {6,0,3,0},
    {4,0,6,0},
    {6,0,5,0},
    {4,0,1,0},
    {4,0,0,2},
    {5,0,6,0},
    {6,0,2,0},
};
```

16.6 Adjacency-list form

Equivalently, the transition relation may be grouped by source state as follows:

$$
\begin{aligned}
&S0[2] \to S7[0],\ S6[2],\\
&S1[0] \to S1[0],\ S4[0],\ S0[2],\\
&S2[0] \to S6[0],\ S4[0],\ S3[0],\ S2[0],\ S5[0],\\
&S3[0] \to S6[0],\ S5[0],\ S3[0],\ S4[0],\ S2[0],\\
&S4[0] \to S5[0],\ S2[0],\ S4[0],\ S3[0],\ S6[0],\ S1[0],\ S0[2],\\
&S5[0] \to S5[0],\ S4[0],\ S2[0],\ S3[0],\ S6[0],\\
&S6[0] \to S6[0],\ S7[0],\ S3[0],\ S5[0],\ S2[0],\\
&S6[2] \to S7[0],\ S6[2],\\
&S7[0] \to S4[0],\ S1[0],\ S0[2].
\end{aligned}
$$

16.7 Key structural property

The transition

$$S1[0] \to S1[0]$$

is present in the compressed relation.

This self-loop is crucial for the proof, as it ensures that the uniform Mode A assignment satisfies all compatibility constraints on the critical core.

16.8 Consistency check

All transitions appearing in the induced core relation (Appendix A) are contained in the compressed edge relation listed above.

This provides an independent consistency check between the local propagator, the compatibility graph Ψ, and the critical-core verification.

References

[1] A. Alkan. On a generalization of hofstadter's q-sequence: A family of chaotic generational structures. *Complexity*, page Article 8517125, 2018.

[2] A. Alkan, N. Fox, and O. O. Aybar. On hofstadter heart sequences. *Complexity*, page Article 2614163, 2017.

[3] B. Balamohan, A. Kuznetsov, and S. Tanny. On the behavior of a variant of hofstadter's q-sequence. *Journal of Integer Sequences*, 10:Article 07.1.2, 2007.

[4] M. Bodirsky and D. Král'. Locally consistent constraint satisfaction problems with binary constraints. In *Graph-Theoretic Concepts in Computer Science*, volume 3787 of *Lecture Notes in Computer Science*, pages 235–246. Springer, 2005.

[5] J. M. Campbell and B. Cloitre. Meta-automatic sequences. *arXiv preprint*, 2026.

[6] M. Celaya and Frank Ruskey. An undecidable nested recurrence relation. In *How the World Computes*, volume 7318 of *Lecture Notes in Computer Science*, pages 138–149. Springer, 2012.

[7] Nathan Fox. Finding linear-recurrent solutions to hofstadter-like recurrences using symbolic computation. *arXiv preprint*, 2016.

[8] Nathan Fox. A new approach to the hofstadter q-recurrence. *arXiv preprint*, 2018.

[9] Nathan Fox. The behavior of a three-term hofstadter-like recurrence with linear initial conditions. *arXiv preprint*, 2024.

[10] Pavol Hell and Jaroslav Nešetřil. *Graphs and Homomorphisms*. Oxford University Press, 2004.

[11] Douglas R. Hofstadter. *Gödel, Escher, Bach: An Eternal Golden Braid*. Basic Books, 1979.

[12] Douglas Lind and Brian Marcus. *An Introduction to Symbolic Dynamics and Coding*. Cambridge University Press, 1995.

[13] Marco Mantovanelli. Dyadic self-similarity in a perturbed hofstadter recursion. *arXiv preprint*, 2026.

[14] Marco Mantovanelli. Verification code for "a finite-state proof of the well-definedness of a perturbed hofstadter sequence". `https://github.com/Mantovanelli1110/perturbed-hofstadter-well-definedness`, 2026. Version corresponding to the paper.

[15] Klaus Pinn. Order and chaos in hofstadter's q-sequence. *Complexity*, 4(3):41–46, 1999.

[16] Stephen M. Tanny. A well-behaved cousin of the hofstadter sequence. *Discrete Mathematics*, 105:227–239, 1992.

[17] P. Trojovský. On some properties of a meta-fibonacci sequence connected to hofstadter sequence and möbius function. *Chaos, Solitons & Fractals*, 134:109719, 2020.

[18] P. Trojovský. On some properties of the hofstadter–mertens function. *Complexity*, page Article 1816756, 2020.

[19] Pavel Trojovský. On properties of hofstadter's q-sequence. *Chaos, Solitons & Fractals*, 133:109644, 2020.